The Evolution of AI Agents and Modern AI Engineering

From Simple AI Programs to Smart, Tool-Using, Multi-Agent Systems

By Maryam Fatima | BS AI Student

Technical Research Report | Prepared July 2026 | Topics: LLM Applications · AI Agents · Tool Calling · Memory · Planning · Multi-Agent Systems · Future Outlook

1. From LLM Applications to Agentic Systems

The first AI apps worked in a simple way. You send one prompt, the AI model gives one answer, and that's it. This is called a **completion pipeline**. Sometimes extra information is added to the prompt first (this is called RAG). The app controls everything, and the model is just used one time per request. This style still works well for simple jobs like sorting text, writing a summary, or pulling out key facts. But it has a big limit — the model cannot take real actions, check its own work, or do a task that needs many steps, unless the app calls it again and again by itself.

AI agents were built to fix this problem. Instead of one call, the model now runs inside a loop. It looks at the current situation, decides what to do next (use a tool, ask a question, or stop), does that action, and checks the result again. This repeats until the goal is done. This is the same idea used in coding tools like Claude Code — the outside program is just a simple manager, and the AI model does the real thinking.

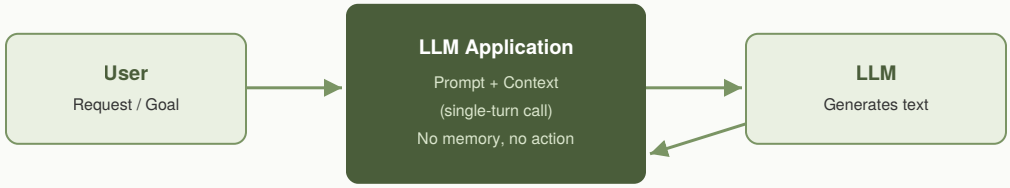

Fig. 1 — A simple AI app: one request, one answer, no real action taken.

2. AI Agents: The Reasoning & Action Loop

An agent puts the AI model inside a repeating loop. People often call this **observe → think → act → observe** (this comes from a method called ReAct). In each round, the model looks at the current chat and situation, decides if it needs a tool, uses that tool if needed, and gets the result back. It uses this result to decide the next step. The loop stops when the model gives a final answer or when a limit is reached (like too many steps, too much time, or too much cost).

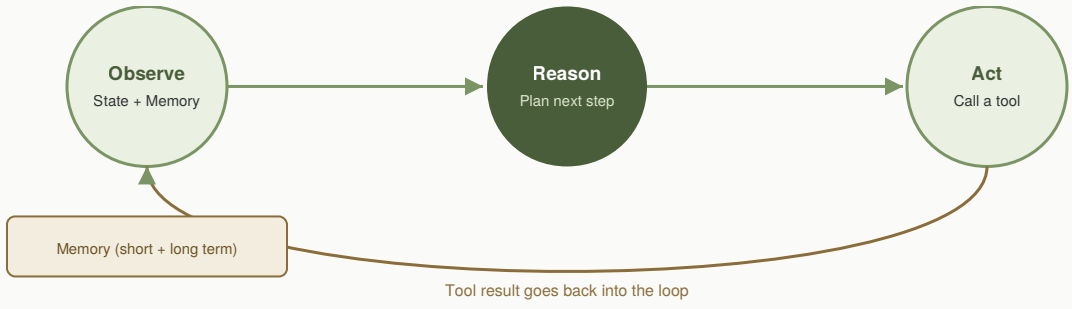

Fig. 2 — The agent loop: thinking, using tools, and memory all work together, again and again.

3. Tool Calling

Tool calling (also called function calling) is what lets an AI model actually do things, not just talk. First, the app tells the model what tools are available (their name, what they do, and what input they need). When the model decides it needs a tool, it does not write plain text — it sends a special structured message asking to use that tool. The app then runs the real tool (like an API call, a database

search, or a command) and sends the result back to the model. The **Model Context Protocol (MCP)** makes this even easier. It gives a common standard way for tools, files, and prompts to connect to any AI agent, so developers do not need to build a new connection for every single app.

> **Real-world example:** A Claude Code helper agent checking Jira tickets can call a tool named `searchJiraIssuesUsingJql` to find open tickets. It reads the results, then calls `transitionJiraIssue` to close a finished ticket — all without a person writing custom code for each ticket.

4. Memory

AI models forget everything after each call. So "memory" is something engineers build around the model, usually in three levels:

Level	What it covers	How it is usually built
Working memory	One chat session	The full chat history kept in the current context
Retrieved memory	Across sessions, by topic	A vector database with search (RAG)
Long-term memory	Facts and preferences that last	A saved file or database, summarized and added back later (like Claude's memory or a project's CLAUDE.md file)

An AI model can only "read" a limited amount of text at once. So it is not possible to just keep adding everything forever. This is why real systems shorten old messages, make summaries, or save things outside the chat and only bring back what is needed. This trades a little bit of detail for lower cost and better focus.

5. Planning

Planning means breaking a big, unclear goal into small, ordered steps before (or while) doing the work. There are two common ways agents plan:

- **Step-by-step planning (ReAct-style):** the model plans only one step at a time, and re-plans after seeing each result. This is easy to build and handles surprises well, but it can get stuck or lose focus on very long tasks.
- **Full planning first:** the model first writes a full task list (like Claude Code's `TodoWrite` feature), then works through it, marking steps done or changing them if needed. This works better for long tasks with many files or many steps, where losing track of the main goal is costly.

> **Real-world example:** When Claude Code is asked to update a software package across four different environments, it first writes a to-do list — check current usage, update the package, run tests, deploy to each environment — and updates this list as each step finishes, instead of figuring it out step by step on the fly.

6. Multi-Agent Systems

When a task gets bigger, one single agent runs into problems — it cannot hold everything in its memory, and it loses focus. Multi-agent systems fix this by splitting the work between several smaller agents, all managed by one **orchestrator** (a lead agent). Each smaller agent has its own memory, its own tools, and its own job. Two common setups are: orchestrator-worker (a lead agent gives tasks to helper agents and then combines their answers), and pipeline (agents work one after another in a fixed order, each one using the last agent's output).

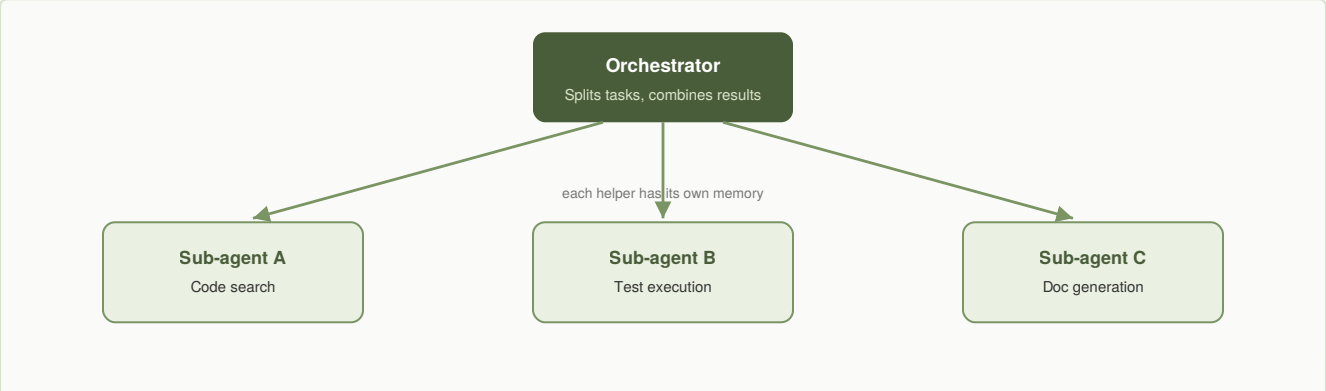

Fig. 3 — Orchestrator-worker setup: each helper agent has its own memory and tools; the lead agent combines all their results.

> **Real-world example:** Anthropic built a multi-agent research system where a lead agent splits a research question into smaller parts. Each helper agent searches for its own part and gives back a short answer. Then the lead agent combines all the short answers into one full report. This works much better than using one single agent for a big research task.

7. Future of AI Engineering

AI engineering is starting to look more like normal software engineering, but for parts that are not always 100% predictable:

- **Common standards** — protocols like MCP make it easier to connect agents with many different tools, without building a new connection each time.
- **Testing AI like code** — using automatic checks (sometimes even another AI as a judge) to test agents before they go live, just like unit tests for normal code.
- **Controlled freedom** — giving agents limits and permissions, and adding human checks for risky actions (like payments or changing servers), instead of letting them do anything.
- **Smart use of models** — using a small, cheap model for easy tasks and a bigger model only for hard reasoning, inside the same workflow, to save time and money.

Overall, AI engineering is moving away from writing one clever prompt, and moving toward building full systems: agents as real products, memory and tools as real infrastructure, and testing as a required step before launch — the same care used in normal software, now used for AI reasoning too.

References

[1] Yao, S. et al. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 2022.

[2] Anthropic. "Building Effective AI Agents." Anthropic Engineering Blog, 2024.

[3] Anthropic. "How we built our multi-agent research system." Anthropic Engineering Blog, 2025.

[4] Anthropic. "Introducing the Model Context Protocol." Anthropic News, 2024.

[5] Anthropic. "Claude Code: Best practices for agentic coding." Anthropic Engineering Blog, 2025.

[6] Schick, T. et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." arXiv:2302.04761, 2023.

[7] Wang, L. et al. "A Survey on Large Language Model based Autonomous Agents." arXiv:2308.11432, 2023.